

PORTADA

Técnicas de Programación

Unidad 7 — Desarrollo Web: Spring Boot, Capas, Persistencia y Cierre de Unidad

Universidad de Antioquia · Ingeniería de Sistemas · 2508307



TRANSICIÓN

De un endpoint suelto a una aplicación organizada

En la sesión anterior aprendiste a exponer datos con Spring Boot: creaste un `@RestController`, mapeaste rutas con `@GetMapping` / `@PostMapping`, y recibiste datos con `@RequestBody` y `@PathVariable`. Pero si toda la lógica —validaciones, cálculos, acceso a datos— vive dentro del controlador, el código no escala: cada método crece sin control y mezcla responsabilidades que no deberían estar juntas.

Esto no es un problema nuevo: es el mismo que resolviste en la **Unidad 3** con el **Principio de Responsabilidad Única** (SOLID) — cada clase debe tener una sola razón para cambiar. Hoy aplicamos ese principio a una aplicación Spring Boot completa, organizándola en **capas**, y cerramos la unidad conectando esa API con una base de datos real.



CONCEPTO

Arquitectura en capas: separar responsabilidades

Una aplicación Spring Boot bien organizada divide el trabajo en tres capas, cada una con una única responsabilidad — exactamente el espíritu del Principio de Responsabilidad Única aplicado a la arquitectura completa:

Capa	Responsabilidad	Anotación típica
Controller	Recibe las peticiones HTTP, valida la forma de la entrada y delega el trabajo real	@RestController
Service	Contiene la lógica de negocio: reglas, cálculos, decisiones de la aplicación	@Service
Repository	Accede a los datos: lee y escribe en la base de datos	@Repository

La petición siempre fluye en un solo sentido: **Controller → Service → Repository**. Un controlador nunca debería hablar directamente con la base de datos.

CONCEPTO

Inyección de dependencias entre capas

Para que un `Controller` use un `Service` (o un `Service` use un `Repository`), no se crea el objeto manualmente con `new`. Spring lo **inyecta** automáticamente: gestiona el ciclo de vida de esos objetos (llamados *beans*) y los entrega listos para usar donde se necesiten.

Existen dos formas: anotar un atributo con `@Autowired`, o —la forma **preferida**— recibir la dependencia como parámetro del constructor, lo que además facilita las pruebas unitarias.

```
@RestController
@RequestMapping("/api/tareas")
public class TareaController {

    private final TareaService tareaService;

    // Inyección por constructor (preferida)
    public TareaController(TareaService tareaService) {
        this.tareaService = tareaService;
    }

    @GetMapping
    public List<Tarea> listar() {
        return tareaService.listarTodas();
    }
}
```

CONCEPTO

@Service y @Repository : marcar cada capa

Para que Spring pueda inyectar un objeto, primero necesita saber que ese objeto existe y debe gestionarlo como *bean*. Por eso cada clase de capa lleva una anotación que la registra:

- `@Service` marca una clase como parte de la capa de lógica de negocio. Spring la detecta automáticamente y la pone disponible para inyectarla en cualquier `Controller` .
- `@Repository` marca una clase (o interfaz) como parte de la capa de acceso a datos. Además, Spring traduce automáticamente ciertas excepciones de base de datos a excepciones propias más claras.

```
@Service
public class TareaService {

    private final TareaRepository tareaRepository;

    public TareaService(TareaRepository tareaRepository) {
        this.tareaRepository = tareaRepository;
    }

    public List<Tarea> listarTodas() {
        return tareaRepository.findAll();
    }
}
```

CONCEPTO

Entidades: una clase Java se vuelve una tabla

Para que la capa `Repository` tenga algo que guardar, primero se necesita una **entidad**: una clase Java simple que, con tres anotaciones, Spring la mapea directamente a una tabla de base de datos, sin escribir SQL para crearla.

- `@Entity` — marca la clase como una tabla.
- `@Id` — marca el atributo que es la clave primaria.
- `@GeneratedValue` — indica que el valor del id se genera automáticamente (por ejemplo, autoincremental).

```
@Entity
public class Tarea {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;
    private boolean completada;

    // getters y setters
}
```

Cada atributo de la clase se convierte en una columna de la tabla; cada objeto `Tarea` guardado, en una fila.

CONCEPTO

Spring Data JPA: persistencia sin escribir SQL

Spring Data JPA es una capa de abstracción sobre JDBC/Hibernate: en vez de escribir manualmente las consultas SQL para guardar, buscar o borrar datos, se define una interfaz que **extiende** `JpaRepository<Entidad, TipoId>` .

```
public interface TareaRepository extends JpaRepository<Tarea, Long> {  
    // sin implementar nada más, ya existen:  
    // findAll(), save(tarea), findById(id), deleteById(id)...  
}
```

Con solo esa línea, Spring genera en tiempo de ejecución una implementación completa con métodos como `findAll()` , `save(Tarea t)` y `findById(Long id)` — sin que el programador escriba ni una sola consulta SQL.

CONCEPTO

Métodos derivados: consultas a partir del nombre

Spring Data JPA va más allá de los métodos básicos: si se declara un método siguiendo una convención de nombres, Spring **genera la consulta automáticamente** a partir de ese nombre, sin necesidad de implementarlo ni escribir SQL.

```
public interface TareaRepository extends JpaRepository<Tarea, Long> {  
  
    // Spring interpreta el nombre y genera:  
    // SELECT * FROM tarea WHERE nombre = ?  
    List<Tarea> findByNombre(String nombre);  
  
    // SELECT * FROM tarea WHERE completada = ?  
    List<Tarea> findByCompletada(boolean completada);  
}
```

El prefijo `findBy` seguido del nombre exacto del atributo (con la primera letra en mayúscula) es suficiente — Spring hace el resto.

CONCEPTO

application.properties : dónde vive la base de datos

Para que Spring Data JPA sepa a qué base de datos conectarse, la aplicación necesita un archivo de configuración mínimo: `src/main/resources/application.properties` . Ahí se declaran la URL de conexión, el usuario y la contraseña de la base de datos.

```
spring.datasource.url=jdbc:mysql://localhost:3306/tareasdb
spring.datasource.username=root
spring.datasource.password=admin
spring.jpa.hibernate.ddl-auto=update
```

No profundizamos aquí en detalles de infraestructura de bases de datos — solo que este es el archivo donde se centraliza esa configuración, separada del código de las capas.

CONCEPTO

Las tres capas juntas: un ejemplo completo

```
@Entity
public class Tarea {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nombre;
    private boolean completada;
    // getters y setters
}

public interface TareaRepository extends JpaRepository<Tarea, Long> { }

@Service
public class TareaService {
    private final TareaRepository repo;
    public TareaService(TareaRepository repo) { this.repo = repo; }
    public List<Tarea> listarTodas() { return repo.findAll(); }
    public Tarea crear(Tarea t) { return repo.save(t); }
}

@RestController
@RequestMapping("/api/tareas")
public class TareaController {
    private final TareaService service;
    public TareaController(TareaService service) { this.service = service; }
    @GetMapping
    public List<Tarea> listar() { return service.listarTodas(); }
    @PostMapping
    public Tarea crear(@RequestBody Tarea t) { return service.crear(t); }
}
```

CONCEPTO

Cerrando el círculo: del `fetch()` a la base de datos

En la **sesión 6** de esta unidad usaste `fetch()` en JavaScript para pedirle datos a una API. Ahora ya sabes qué hay del otro lado de esa petición: una aplicación Spring Boot completa, organizada en capas.

```
fetch("/api/tareas")
  .then(respuesta => respuesta.json())
  .then(tareas => console.log(tareas));
```

El recorrido completo de esa única línea de `fetch()` es: el navegador envía la petición HTTP → el **Controller** la recibe y la delega → el **Service** aplica la lógica de negocio → el **Repository** consulta la base de datos → los datos regresan como JSON hasta el `fetch()` que los pidió. Frontend y backend, unidos por HTTP.

SÍNTESIS DE LA UNIDAD 7

El viaje completo: de una etiqueta HTML a una API con base de datos

1. **HTML** — estructura y semántica: etiquetas, formularios, accesibilidad, la base sobre la que se construye cualquier página.
2. **CSS3** — estilo y layout: box model, Flexbox, Grid y diseño responsivo para que esa estructura se vea bien en cualquier pantalla.
3. **JavaScript** — interactividad: manipulación del DOM, eventos, funciones de orden superior y `fetch()` para que la página deje de ser estática y hable con un servidor.
4. **Spring Boot** — la API del otro lado: `@RestController` para exponer endpoints, y arquitectura en capas (`Controller` → `Service` → `Repository`) con Spring Data JPA para persistir datos reales.

Ocho sesiones después, ya puedes construir una aplicación web completa: una interfaz que el usuario ve y con la que interactúa, conectada por HTTP a un backend que organiza su lógica y guarda su información en una base de datos.



CIERRE DE UNIDAD 7

Antes del Laboratorio Final

La Unidad 7 se evalúa con un **Laboratorio Final (20%)** que integra lo visto en las 8 sesiones: HTML, CSS3, JavaScript y Spring Boot con persistencia. El enunciado y los entregables se presentan aparte, en la sesión de laboratorio.

Repasa antes de esa evaluación:

- Estructura semántica en HTML y estilos responsivos con CSS3 (Flexbox/Grid).
- Manipulación del DOM, eventos y `fetch()` en JavaScript.
- Arquitectura en capas de Spring Boot y cómo cada anotación (`@RestController` , `@Service` , `@Repository` , `@Entity`) conecta esas capas entre sí.
- Spring Data JPA: qué obtienes gratis al extender `JpaRepository` y cómo declarar un método derivado.

